

Week 8: Sift DSL: Architecture, Control Flow, and Design Reflection

Aaron Foster

CS 660: Advanced Programming Language Theory

Southern New Hampshire University

Shadha Tabatabai

August 29th, 2026

Part One: Architecture and Control Flow Diagram

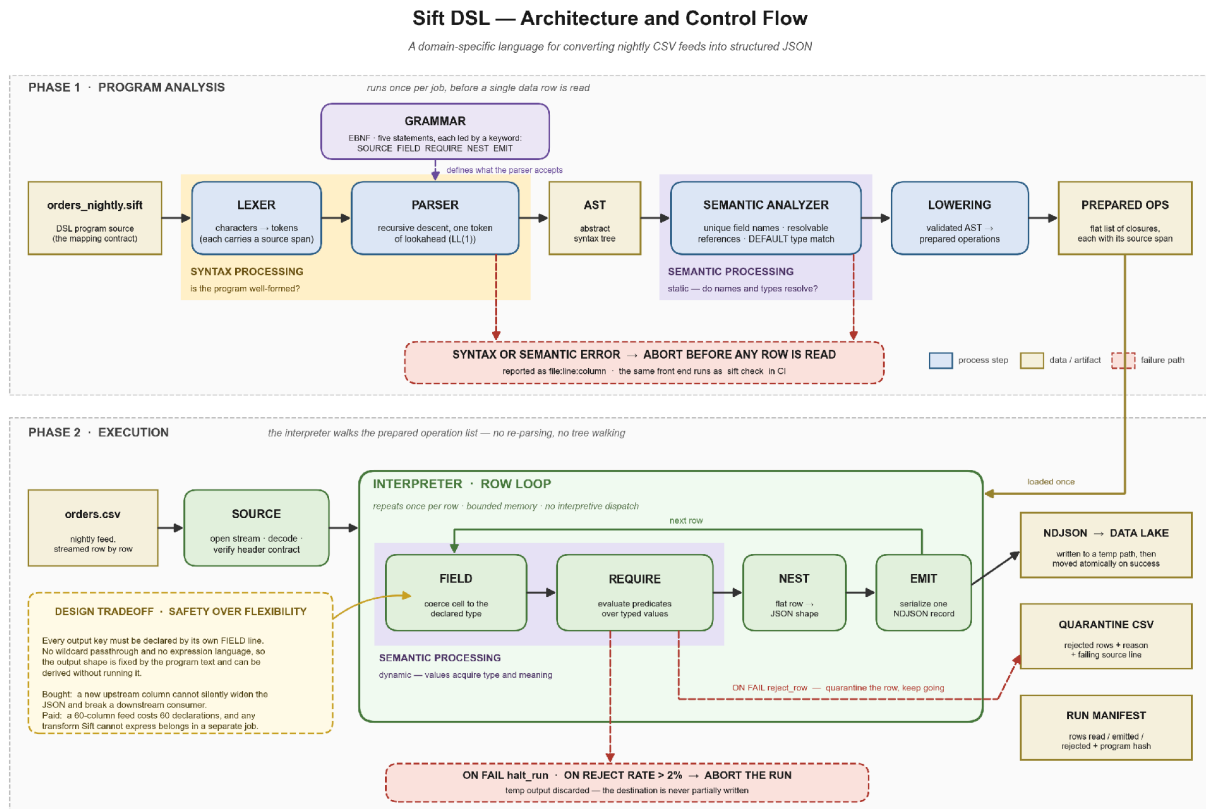


Figure 1. Architecture and control flow of the Sift DSL, a language for converting nightly CSV feeds into structured JSON.

Part Two: Reflection

System Structure

Sift runs in two phases, shown as the two lanes of the diagram. Phase one reads the program text: the lexer turns characters into tokens, the parser assembles them into a syntax tree under the rules of the grammar, the semantic analyzer validates names and types, and a lowering pass compiles that tree into a flat list of prepared operations. Phase two reads the data: SOURCE opens the CSV as a stream and verifies the header contract, then the interpreter walks the prepared operations once per row through FIELD, REQUIRE, NEST, and EMIT.

Syntax and Semantics

Syntax is handled entirely in phase one and is defined by the grammar, an EBNF description of five statements, each led by a distinct keyword. One token of lookahead identifies every production, so the recursive-descent parser can reject a malformed program and name the offending line and column. Semantics are handled in two places, marked by the shaded bands. Static semantics live in the analyzer: names must be unique, references must resolve, and DEFAULT literals must match declared types. Dynamic semantics live in the row loop, where FIELD coerces raw text into declared types and REQUIRE evaluates predicates over the results. Every meaning error catchable without data is caught before the run starts.

Rationale for the Tradeoff

The annotated tradeoff is safety over flexibility. Every output key must be declared by its own FIELD line, with no wildcard passthrough and no expression language, so a sixty-column feed costs sixty declarations. The benefit is that the output shape is fixed by the program text and derivable without executing it, so a new upstream column cannot silently widen the JSON. The asymmetry justifies the choice: verbosity is paid once by an engineer at authoring time, while a silent schema change is paid every night by whoever is on call.

Evaluating the Diagram

The diagram communicates control flow well. The two lanes make the once-per-job and once-per-row distinction visible at a glance, the arrows give an unbroken path from source text to written output, and rendering failures separately shows that aborting, quarantining a row, and halting a run are three distinct outcomes rather than one generic error. The bands also show that semantic processing happens twice, which prose struggles to convey. Its limits are equally clear: it shows the order of operations but not their cost, so nothing conveys that the row loop runs

twenty million times while phase one runs once, and lowering is flattened into one box that hides what makes staged interpretation fast. The diagram establishes what happens and in what order, not whether it is fast enough.